

作业1

8. 除计算最大公约数问题外，列出一个你已知有多种算法的问题。其中哪个算法更简单？哪个算法效率更高？

答：

排序问题

解决方法有：

冒泡排序:

- 它通过多次遍历待排序的数列，依次比较相邻的两个元素，如果顺序错误就交换它们，直到整个序列有序。
- 算法复杂度：平均时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(1)$ 。

选择排序

- 每次选择数组中最小（或最大）的元素，放到已排序的部分的末尾。
- 时间复杂度： $O(n^2)$, 空间复杂度： $O(1)$ 。

插入排序 (Insertion Sort) :

- 将数组分为已排序和未排序两部分，每次从未排序部分取出一个元素，插入到已排序部分的正确位置。
- 算法复杂度：时间复杂度： $O(n^2)$, 空间复杂度： $O(1)$ 。

归并排序 (Merge Sort) :

- 采用分治法，将数组分成两半，分别排序后再合并。
- 算法复杂度：时间复杂度： $O(n \log n)$, 空间复杂度： $O(n)$ （需要额外的空间来合并）

快速排序:

- 利用堆的性质进行排序，将数组构建成最大堆（或最小堆），然后反复将堆顶元素取出，直到整个数组有序。
- 算法复杂度：时间复杂度： $O(n \log n)$ 平均情况， $O(n^2)$ 最坏情况, 空间复杂度： $O(\log n)$

堆排序:

- 它通过多次遍历待排序的数列，依次比较相邻的两个元素，如果顺序错误就交换它们，直到整个序列有序。
- 算法复杂度：时间复杂度： $O(n \log n)$, 空间复杂度： $O(1)$

1. 简单性：

- 冒泡排序、选择排序、插入排序通常被认为是比较简单的排序算法，因为它们的实现相对直观和易理解。
- 归并排序和快速排序虽然较为复杂，但由于它们是分治算法，因此实现也可以相对清晰。

2. 效率：

- 在大多数情况下，快速排序和归并排序通常被认为是比较高效的排序算法，尤其是在平均情况下。
- 冒泡排序、选择排序、插入排序在大规模数据上的性能相对较差，因为它们的时间复杂度较高。

作业2

题目：目前只能用蛮力测略算法求解的问题举例(不能举教材上的例子)。

答：

1. 子集和问题：

- 问题描述：给定一组正整数，判断是否存在其中的某些数的和等于给定的目标值。
- 蛮力求解：尝试列举所有可能的子集，计算每个子集的和，检查是否有子集的和等于目标值。

2. 排列组合问题：

- 问题描述：给定一组元素，求其所有可能的排列或组合。
- 蛮力求解：穷举所有可能的排列或组合，生成所有可能的情况。

3. 图的同构性问题 (Graph Isomorphism Problem)：

- 在图论中，判断两个图是否同构 (具有相同的结构)。目前尚未找到多项式时间内解决这个问题的算法，因此对于一些复杂的图结构，仍然需要蛮力策略。

作业3

“改变表现”和“问题化简”分别举例(不能举教材上的例子)

答：

改变表现 (Change of Representation)：

改变问题的表现方式，有时可以使问题更容易理解或者更容易应用某种特定的算法。

问题：N 皇后问题

在标准的 N 皇后问题中，要求在 $N \times N$ 的棋盘上放置 N 个皇后，使得它们互相之间不能攻击。这个问题的原始表现形式可能比较复杂，但可以通过改变表现来简化问题。

改变表现：位运算表示棋盘状态

将每一行、每一列、以及两条主对角线和两条副对角线分别用二进制数表示。这样，整个棋盘状态可以用一个 N 位的二进制数表示。通过位运算，可以有效地检查是否存在冲突，从而简化了问题的求解过程。

问题化简 (Problem Reduction)：

问题化简是将原始问题转化为一个与之等价但更容易解决的问题。以下是一个例子：

问题：单向TSP旅行商问题

旅行商问题要求寻找一条路径，使得旅行商能够经过每个城市一次，总行程最短。这是一个经典的组合优化问题。

问题化简：最小生成树问题

将旅行商问题化简为求解最小生成树问题。可以通过找到城市之间的最小生成树，然后在最小生成树上进行遍历来解决旅行商问题。这种问题化简使得原问题的求解过程更加简单，因为最小生成树问题有更为高效的解

法。

这两种策略都是为了使问题更易于理解和解决，通过不同的角度或者转换，能够提供更直观或者更高效的方法来处理复杂的问题。

作业4

“输入增强”举例(不能举教材上的例子)；

答：

例子：图像分类中的数据增强

在图像分类任务中，为了增强模型的性能，可以对输入的图像进行各种变换，以生成更多样化的训练样本。这有助于提高模型的泛化能力，使其在面对不同角度、光照条件或者尺寸的图像时表现更好。

例如，对于一张狗的图像，可以进行以下数据增强操作：

- 旋转：将图像旋转一定角度，模拟不同拍摄角度。
- 翻转：水平或垂直翻转图像，增加镜像样本。
- 缩放：调整图像尺寸，模拟不同距离拍摄的效果。
- 平移：在图像上进行平移操作，改变物体的位置。

通过这些操作，可以生成多个变换后的图像，扩充训练数据集，提高模型对于输入变化的适应性。

输入增强在深度学习中广泛应用，不仅限于图像分类，还包括其他领域，如语音识别、自然语言处理等。

作业五

动态规划法和分治法有什么共同点?这两种技术之间最主要的不同点是什么?

答：

共同点：

1. **问题分解**：动态规划法和分治法都采用问题分解的思想，将原始问题划分为更小的子问题，然后分别解决这些子问题。
2. **递归结构**：两者都具有递归结构，即问题的解取决于其子问题的解。通过递归地解决子问题，最终得到原始问题的解。

主要不同点：

1. **问题重叠性**：
 - **动态规划**：解决子问题时将解存储起来，以避免重复计算。动态规划算法通常用于解决具有重叠子问题性质的问题，通过记忆化或自底向上的方法避免重复计算。
 - **分治法**：每个子问题只求解一次，不保留子问题的解。分治法通常用于解决问题的子问题不重叠的情况。
2. **子问题关系**：

- **动态规划**：子问题之间可能存在相互依赖的关系，解一个子问题可能会影响其他子问题的解。动态规划常用于具有最优子结构性质的问题，即问题的最优解可以由其子问题的最优解组合而成。
- **分治法**：子问题之间相互独立，每个子问题的解都是独立计算的。分治法通常应用于问题可以被划分为互不相交的子问题的情况。

3. 最终结果组合：

- **动态规划**：解决子问题时保存子问题的解，最终通过组合这些子问题的解得到原始问题的解。
- **分治法**：子问题的解不保存，最终问题的解是通过组合各个子问题的解而得到。

总的来说，动态规划和分治法在思想上有相似之处，都是通过问题分解和递归求解子问题。主要区别在于动态规划对子问题的解进行了存储，以避免重复计算，并且子问题之间可能存在相互依赖的关系。

作业六

“贪心算法”举例 (不能举教材上的例子)

答：

例子：找零问题 (Coin Change Problem)

假设你是一位售货员，需要找零 n 元钱给顾客，而你手上有 1 元、2 元和 5 元的硬币。请问，如何使用最少数量的硬币找零？

(

贪心策略：
1. 优先使用面值较大的硬币，因为它们相对更少，能够更快达到找零的最小数量。
2. 重复上述步骤，直到完全找零为止。

)

举例说明：

假设 n = 13 元，可用的硬币面值为 1 元、2 元和 5 元。

1. 优先使用 5 元硬币，得到 5 元，余额 $13 - 5 = 8$ 元。
2. 优先使用 5 元硬币，得到 5 元，余额 $8 - 5 = 3$ 元。
3. 优先使用 2 元硬币，得到 2 元，余额 $3 - 2 = 1$ 元。
4. 使用 1 元硬币，得到 1 元，余额 $1 - 1 = 0$ 元。

总共使用了 4 枚硬币，包括两枚 5 元硬币、一枚 2 元硬币和一枚 1 元硬币，实现了最小数量的找零。这是因为贪心算法每一步都选择当前状态下的最优解，尽可能使用面值大的硬币。需要注意，贪心算法并不总是能够得到全局最优解，但在某些问题上却表现出令人满意的效果。